

P R O J E C T P O R T F O L I O

RealtyAgent

부동산·법률 특화 멀티에이전트 AI 시스템

LangGraph · FastAPI · LlamaIndex · Qdrant · Neo4j · Redis · GLiNER

1. 프로젝트 소개 및 문제 정의

1-1. 한 줄 소개

RealtyAgent는 부동산 및 법률 상담에 특화된 멀티에이전트 AI 시스템으로, 사용자의 복잡한 질문을 분석하고 법적 근거·실거래 데이터·개인 맥락을 통합하여 구조화된 상담 리포트를 자동 생성합니다.

1-2. 사용자 시나리오

처음 전세 계약을 앞둔 사람을 예로 들면, 실질적으로 아래와 같은 질문들을 동시에 해결해야 합니다.

- 이 집에 은행 대출이 얼마나 걸려 있어서, 내 전세금이 위험한 건 아닐까? (선순위 채권 확인)
- 집주인이 세금을 밀리고 있으면 내 보증금이 날아갈 수도 있는데, 어떻게 확인하지? (임대인 세금 체납 여부)
- 계약서에 '특약사항'이 있는데, 이게 나한테 불리한 조항인지 아닌지 모르겠다. (법적 유효성 판단)

이 세 가지 질문은 각각 등기부등본(공문서), 주택임대차보호법(법령), 실거래 사례(데이터)를 따로 찾아봐야 답이 나옵니다. 일반 검색이나 단일 챗봇으로는 이 과정을 한 번에 처리할 수 없으며, 사용자가 직접 여러 출처를 돌아다니며 정보를 조합해야 합니다. RealtyAgent는 이 과정을 하나의 질문으로 처리하는 것을 목표로 합니다.

1-3. 기존 접근법의 한계

접근법	할 수 있는 것	한계
포털 검색	정보 단편 수집	여러 출처를 사용자가 직접 조합해야 함
법무사·공인중개사	전문적 판단	비용 발생, 즉각 응답 어려움

1-4. 핵심 도전 과제

"법령 근거 + 실거래 데이터 + 사용자 개인 맥락"을 단일 흐름에서 동적으로 조합하고, 각 단계마다 품질·안전성을 보장하는 프로덕션 수준의 에이전트 시스템 설계

이를 해결하기 위해 LangGraph 기반의 유향 그래프(DAG) 워크플로우를 채택하였고, 에이전트 실행 로직과 인프라를 완전히 분리하는 인터페이스 설계를 적용했습니다.

2. 시스템 아키텍처

2-1. 레이어 구성

시스템은 4개 레이어로 구성됩니다.

레이어	구성 요소	역할
API Gateway	FastAPI (server/)	인증, 세션 관리, 렌더링, 요청 라우팅
Agent Engine	LangGraph (engine/)	멀티에이전트 워크플로우 실행, 상태 관리
Storage / Worker	PostgreSQL · Qdrant · Redis · Neo4j	영속 저장, 벡터 검색, 비동기 태스크

2-2. 핵심 설계 원칙: ExternalDepsPort

이 시스템에서 가장 중요한 아키텍처 결정은 **engine/**과 인프라 간의 완전한 분리입니다.

engine/(LangGraph)은 PostgreSQL, Qdrant, LlamaIndex 등 구체적인 구현을 전혀 알지 못합니다. 오직 ExternalDepsPort Protocol 인터페이스만을 통해 의존성을 주입받습니다.

인터페이스 메서드	실제 구현 (server/)	연결 저장소
search_memories()	ExternalDepsService	PostgreSQL (단기 대화 이력)
search_docs()	ExternalDepsService	ingestion/ → Qdrant + Neo4j
get_user_persona()	ExternalDepsService	PostgreSQL (GLiNER 프로필)
push_task()	ExternalDepsService	Redis Queue

이 구조의 실질적 이점은 두 가지입니다.

- 검색 전략(Vector → GraphRAG 전환 등)이나 저장소 교체 시 **engine/** 코드 수정 불필요
- 엔진 로직과 인프라를 각각 독립적으로 테스트 가능 (인터페이스 Mock 주입)

3. 에이전트 워크플로우 설계

3-1. LangGraph DAG 선택 이유

선형 파이프라인($A \rightarrow B \rightarrow C \rightarrow D$)은 조건부 제시도, 동적 실행 경로, 인간 개입 지점을 표현할 수 없습니다. LangGraph는 노드와 �지를 유향 그래프로 정의하고 런타임에 라우팅을 결정할 수 있어, 복잡한 에이전트 워크플로우에 적합합니다.

3-2. Planner → Dispatcher 패턴

사용자 질문이 입력되면 Planner가 의도를 분석하여 `node_stack`(실행 노드 순서 리스트)를 생성합니다. Dispatcher는 이 스택을 순차적으로 소비하며 다음 노드로 라우팅합니다.

Planner 출력 필드	설명	예시
<code>intention</code>	사용자 의도 한 문장 요약	임대차 계약서 작성 요청
<code>refined_query</code>	검색 최적화된 쿼리	전세계약 확정일자 주택임대차보호법
<code>node_stack</code>	실행할 노드 순서 리스트	[<code>counselor</code> , <code>legal_retriever</code> , <code>generator</code>]

설계 의도: Planner가 `node_stack`만 결정하고, 실제 실행 제어는 Dispatcher에 위임함으로써 계획(What)과 실행(How)을 분리했습니다. Planner가 지정할 수 있는 노드는 `counselor`, `legal_retriever`, `doc_retriever`, `memory_retriever`, `generator`, `human_reviewer` 6개이며, 시스템 노드(`Evaluator`, `Finalizer` 등)는 자동 관리합니다. 문서 작성이 필요한 요청에는 `counselor`를 `generator` 앞에 배치하여 필요 정보를 사전 수집합니다.

3-3. 전체 노드 역할

노드	역할	비고
Initializer	세션 초기화, PostgreSQL 대화 이력 로딩	자동 실행
Planner	의도 분석 → <code>node_stack</code> 생성	자동 실행
Dispatcher	<code>node_stack pop</code> → 다음 노드 라우팅	자동 실행
Counselor	문서 작성 전 필요 정보 수집 (<code>interrupt_after</code> , <code>self-loop</code>)	Planner 지정
LegalRetriever	법령·판례 검색 + <code>self-loop</code> 검증 (Circuit Breaker 내장)	Planner 지정
DocRetriever	매물·실거래가·공문서 검색 + <code>self-loop</code> 검증 (Circuit Breaker 내장)	Planner 지정
MemoryRetriever	과거 대화 이력·GLINER 프로필 조회 (재시도 없음)	Planner 지정

노드	역할	비고
Generator	document_type별 YAML 템플릿 + retrieved_docs 기반 결과 생성	Planner 지정 / 자동 실행
Evaluator	보안·환각(Optional)·개인정보 3중 품질 검사	자동 실행
HumanReviewer	중간 승인 요청 (interrupt_before, REPLAN / REWRITE / APPROVE)	Planner 지정
Finalizer	최종 결과 정리, Redis Worker 태스크 발행	자동 실행

3-4. 주요 설계 포인트

Retriever Self-loop — Circuit Breaker 내장

LegalRetriever와 DocRetriever는 검색 완료 후 내부에서 결과 유효성을 직접 검증합니다(doc 길이 + 프롬프트 인젝션 검사). 검증 실패 시 자기 자신으로 재시도하는 **self-loop** 구조이며, 3회 초과 시 강제로 Dispatcher로 복귀합니다. MemoryRetriever는 실패 시 빈 값으로 처리하고 재시도 없이 항상 Dispatcher로 복귀합니다.

Retriever 조건	다음 동작
검증 실패 + 재호출 횟수 < 3	자기 자신으로 재시도 (self-loop)
검증 통과	Dispatcher 복귀
재호출 횟수 >= 3 (한도 초과)	강제로 Dispatcher 복귀

Counselor — interrupt_after 기반 정보 수집

문서 작성이 필요한 요청에서 Generator 실행 전 사용자와 대화하며 필수 정보를 한 항목씩 수집합니다. interrupt_after로 동작하여 실행 직후 그래프가 일시 중단되고, resume(feedback)으로 사용자 답변이 주입되면 다음 턴이 실행됩니다. 필수 정보가 모두 수집되면(is_ready=True) Dispatcher로 복귀하여 다음 노드로 진행합니다.

문서 유형	필수 수집 항목
legal_report	상담 주제, 분쟁 상황, 관련 당사자
lease_contract	임대인/임차인 정보, 부동산 소재지, 보증금, 임대료, 임대 기간
sale_contract	매도인/매수인 정보, 부동산 소재지, 매매 대금, 계약금·중도금·잔금 일정
legal_memo	발신인/수신인 정보, 작성 목적, 관련 사실 관계

Generator — document_type별 YAML 템플릿

Generator는 ConsultationContext의 document_type을 런타임에 읽어 해당 YAML 프롬프트 템플릿을 동적으로 로딩합니다. retrieved_docs와 consultation_context를 결합하여 문서 유형에 맞는

결과물을 생성합니다.

YAML 파일	문서 유형	설명
chat.yaml	chat	가벼운 대화형 응답 (기본값)
legal_report.yaml	legal_report	법률 상담 리포트
lease_contract.yaml	lease_contract	임대차 계약서
sale_contract.yaml	sale_contract	매매 계약서
legal_memo.yaml	legal_memo	내용증명 / 법률 메모

Evaluator — 3중 품질 검사

Generator가 결과물을 생성하면 Evaluator가 세 가지 항목을 검사합니다. 하나라도 실패 시 Generator를 재시도하거나 HumanReviewer로 에스컬레이션합니다.

검사 항목	설명	실패 시 처리
is_secured	프롬프트 인젝션 공격 탐지	Generator 재시도 또는 HumanReviewer
is_grounded	환각(Hallucination) 검증 (Optional)	Generator 재시도
has_pii	개인정보 포함 여부 탐지	Generator 재시도 또는 HumanReviewer

HumanReviewer — interrupt_before 기반 Human-in-the-Loop

interrupt_before로 동작하여 노드 실행 전 그래프가 일시 중단되고 사용자 피드백을 기다립니다. resume(feedback)으로 주입된 텍스트를 LLM이 분석하여 아래 세 가지 액션으로 분류합니다.

피드백 액션	라우팅 대상	설명
REPLAN	Planner	실행 전략 자체를 재수립
REWRITE	Generator	현재 검색 결과 기반 답변 재작성
APPROVE	Dispatcher	승인 후 다음 단계 계속 진행

3-5. AgentState 주요 필드

필드	타입	설명
messages	List[BaseMessage]	전체 대화 메시지 이력
query	str	원본 사용자 질문

필드	타입	설명
planner_response	PlannerResponse	Planner 출력 (node_stack, refined_query, intention)
next_node	NodeType	Dispatcher가 라우팅할 다음 노드
retrieved_docs	dict	각 Retriever가 수집한 검색 결과
consultation_context	ConsultationContext	Counselor가 턴마다 누적하는 상담 정보 (문서 유형 + 동적 필드)
user_input	str	단일 입력 채널. resume()으로 주입, Counselor·HumanReviewer 공유
counselor_question	str	Counselor가 프론트엔드에 전달하는 현재 질문 (answer와 별도 채널)
human_feedback	HumanFeedback	HumanReviewer 내부에서 생성된 피드백 분류 결과
evaluation_response	EvaluationResponse	Evaluator 검사 결과
circuit_check	CircuitCheck	노드별 재호출 횟수 추적
answer	str	Generator 최종 생성 결과 (counselor_question과 분리)
retry_count	int	재시도 횟수

4. 문서 수집 및 검색 파이프라인 (ingestion/)

4-1. 파이프라인 설계 개요

ingestion/ 패키지는 PDF 공문서를 수집하여 검색 가능한 형태로 저장합니다. LlamaIndex 프레임워크를 사용하며, ExternalDepoPort 인터페이스를 경계로 engine/(LangGraph)과 완전히 분리됩니다.

engine/은 LlamaIndex를 전혀 알지 못합니다. DocRetriever 노드는 ExternalDepoPort.search_docs() 만 호출하며, 검색 전략이 바뀌어도 engine/ 코드는 수정되지 않습니다.

4-2. 수집 파이프라인 4 단계

단계	처리 내용	사용 컴포넌트
① 문서 로딩	PDF 파일 로딩	SimpleDirectoryReader (LlamaIndex)
② 청킹	텍스트 분할 (512 토큰, 50 overlap)	SentenceSplitter
③ 개체/관계 추출	LLM 추출기 + 규칙 추출기 병렬 실행	kg_extractors (동시 주입)
④-A Qdrant 저장	벡터 임베딩 저장	QdrantVectorStore [documents]
④-B Neo4j 저장	개체/관계 그래프 저장	Neo4jPropertyGraphStore

4-3. 이중 추출기 — 정확도와 비용의 트레이드오프

개체/관계 추출은 LLM 추출기와 규칙 추출기를 동시에 실행하여 결과를 합산합니다. 두 방식은 서로 다른 강점과 약점을 보완합니다.

구분	LLM 추출기	규칙 추출기
기반	SimpleLLMPATHExtractor (gpt-4o-mini)	TransformComponent (정규식 + 키워드 사전)
추출 방식	문맥 이해 기반 자유 트리플	부동산 도메인 특화 패턴 매칭
강점	복잡한 문맥, 예상치 못한 관계 파악	정확도 높음, API 비용 없음
약점	API 비용 발생, 환각 가능성	사전 정의된 패턴만 인식
추출 대상	자유로운 (주체, 관계, 대상) 트리플	법령명·지역명·건물유형·면적·가격

설계 판단: LLM만 사용하면 비용과 환각 위험이 있고, 규칙만 사용하면 표현력이 부족합니다. 두

추출기를 병렬 실행하여 도메인 특화 정확도(규칙)와 표현 풍부성(LLM)을 동시에 확보했습니다.

4-4. 하이브리드 검색 — Vector + Graph

두 검색기를 결합한 하이브리드 전략을 채택했습니다.

검색기	저장소	역할
VectorContextRetriever	Qdrant	질문과 의미적으로 유사한 청크 Top-K 반환
LLMSynonymRetriever	Neo4j	동의어·관련 개체 경로 그래프 탐색

두 검색기의 결과는 PropertyGraphIndex.as_retriever()에 의해 병합되어 DocRetriever에 최종 컨텍스트로 전달됩니다.

4-5. LlamaIndex 선택 근거

필요 기능	LlamaIndex 제공 컴포넌트
PDF 로딩	SimpleDirectoryReader 내장 지원
청킹	SentenceSplitter 등 다양한 전략
Qdrant 연동	QdrantVectorStore 공식 지원
Neo4j 연동	Neo4jPropertyGraphStore 공식 지원
GraphRAG	PropertyGraphIndex 성숙한 구현체
하이브리드 검색	VectorContextRetriever + LLMSynonymRetriever 내장

5. 데이터 저장소 설계 전략

5-1. 정보 출처 기반 저장소 분리 원칙

단일 저장소에 모든 데이터를 넣는 대신, 정보의 출처와 검색 특성에 따라 저장소를 분리하는 원칙을 적용했습니다.

정보 출처	특성	적합한 저장소
사용자 발화	동적, 개인적, 순서 보장 필요, 비정형	PostgreSQL (관계형)
공문서	정적, 공개, 의미 기반 검색 필요	Qdrant (Vector DB) + Neo4j

5-2. 저장소별 역할과 선택 근거

PostgreSQL — 대화 이력 + 사용자 프로필

대화 이력은 시간 순서(순서 보장)가 핵심이므로 관계형 DB를 선택했습니다. 사용자가 대화 중 언급한 조건(지역, 예산, 평형 등)은 GLiNER 엔티티 추출기가 구조화하여 JSON 프로파일로 누적 갱신합니다. 이를 통해 이후 세션에서도 사용자 조건을 자동으로 맥락 보정합니다.

Qdrant (Vector DB) — 공문서 RAG + 사용자 메모리

공문서 청크는 임베딩 후 [documents] 컬렉션에 저장하여 RAG 유사도 검색에 활용합니다. 사용자 발화 장기 의미 검색은 [user_memory] 컬렉션으로 반드시 분리하여 운영합니다. 컬렉션 분리는 데이터 오염 방지와 검색 정확도 유지를 위한 필수 설계입니다.

Redis — 비동기 태스크 큐

Finalizer 노드는 워크플로우 완료 후 GLiNER 엔티티 추출 등 후처리 태스크를 Redis 큐에 발행합니다. Worker가 이를 비동기로 처리함으로써 응답 지연 없이 후처리를 분리합니다.

Neo4j — GraphRAG (관계 추론, 후순위)

공문서 내 개체 간 관계망(예: 법령 → 규정 대상 → 적용 지역)을 그래프로 저장합니다. 현재는 Vector 검색과 병행하는 하이브리드 구성이며, 다단계 관계 추론이 더 필요해질 경우 GraphRAG 비중을 높이는 방향으로 확장합니다.

5-3. 맥락 보정 실행 흐름

사용자가 조건을 생략하거나 불완전한 질문을 한 경우, 4단계로 정보를 수집하여 프롬프트를 풍부하게 구성합니다.

단계	소스	수집 내용	적용 범위
①	PostgreSQL 대화 이력	최근 N턴 원본 대화	단기 맥락
②	PostgreSQL 사용자 프로 필	GLiNER 누적 엔티티	장기 조건 보정
③	Qdrant [documents]	관련 공문서 청크	도메인 지식
④ (Optional)	Qdrant [conversations]	의미 기반 과거 발화	서술형 장기 맥락

Optional 항목 현재 미도입 이유: GLiNER 구조화 추출로 대부분의 조건 보정이 가능합니다. '조용한 환경 선호'처럼 구조화할 수 없는 서술형 의견이 검색 요구 조건이 되는 시점에 도입을 검토합니다. 현재 시점에서의 미도입은 기능 부족이 아닌 의도적 트레이드오프 결정입니다.

6. API 설계

6-1. 엔드포인트 구성

모든 엔드포인트는 JWT 인증이 필요하며, 추론 관련 엔드포인트는 SSE(Server-Sent Events) 스트리밍으로 응답합니다.

메서드	경로	설명
POST	/new	새 대화 시작 (thread_id 자동 생성)
POST	/{{thread_id}}	기존 대화 이어서 질문
POST	/{{thread_id}}/resume	HumanReviewer 대기 워크플로우 재개
GET	/{{thread_id}}/state	현재 에이전트 상태 조회
GET	/{{thread_id}}/download	최종 리포트 HTML 파일 다운로드

6-2. SSE 스트리밍 응답 형식

추론 엔드포인트는 노드 실행 진행 상황 및 최종 결과를 SSE(Server-Sent Events) 스트림으로 순차 전달합니다. 각 이벤트는 HTML 렌더링된 체크 형태로 전송됩니다.

6-3. /resume 엔드포인트 설계 의도

HumanReviewer 노드는 사용자 피드백을 기다리며 워크플로우를 일시 중단합니다. 기존 대화 엔드포인트(/{{thread_id}})와 분리된 /resume 경로를 둬으로써, 일반 질문 흐름과 승인 흐름을 명확히 구분합니다. 엔진 내부에서 피드백 텍스트를 분석하여 REPLAN / REWRITE / APPROVE 액션으로 자동 분기합니다.

6-4. 출력 형식

Generator는 구조화된 JSON 리포트를 생성하며, Jinja2 렌더러를 통해 HTML 및 PDF로 변환됩니다.

JSON 필드	내용
report_meta	제목, 요약 등 메타 정보
key_issues	주요 쟁점 목록
legal_grounds	법적 근거 목록
practical_advice	실질적 조언 목록

JSON 필드	내용
precautions	주의사항 목록

6-5. 문서 수집 API

공문서(PDF)를 수집하여 RAG 파이프라인에 등록하는 엔드포인트입니다. 모든 요청에 JWT 인증이 필요하며, PDF 이외의 파일은 400 오류로 거부됩니다.

메서드	경로	설명
POST	/documents/upload	PDF 단건 업로드 → 청킹·임베딩·저장
POST	/documents/upload/batch	PDF 다건 업로드 → <code>asyncio.gather</code> 병렬 처리
POST	/documents/scan	서버 DOCS_DIR 경로의 PDF 전체 일괄 처리

단건 업로드는 처리 완료 후 저장된 청크 수를 반환합니다. 다건 업로드는 파일별 처리 결과 (`chunks_stored`, `status`)와 전체 합산(`total_files`, `total_chunks`)을 함께 반환하며, 개별 파일 오류가 발생해도 나머지 파일 처리는 계속됩니다. 스캔 엔드포인트는 `DOCS_DIR` 환경변수가 미설정된 경우 400 오류를 반환합니다.

7. 기술적 고민과 배운 점

7-1. 설계하면서 고민했던 지점

① 프레임워크 혼용 문제 (LangGraph + LlamaIndex)

에이전트 엔진은 LangGraph, 문서 수집은 LlamaIndex를 사용합니다. 두 프레임워크를 경계 없이 혼용하면 관심사가 분리되지 않아 어느 한 프레임워크를 교체하거나 검색 전략을 변경할 때 연쇄적인 코드 수정이 발생합니다. ExternalDepsPort 인터페이스를 경계로 두 프레임워크를 격리함으로써 각 레이어를 독립적으로 변경·테스트할 수 있는 구조를 확보했습니다.

② 재시도 루프의 무한 루프 방지

Verifier와 Evaluator에서 실패 시 재시도하는 구조는 잘못 설계하면 무한 루프에 빠질 수 있습니다. Verifier는 AgentState의 circuit_check 필드에 노드별 재호출 횟수를 기록하여 3회 초과 시 강제로 Dispatcher에 복귀하는 Circuit Breaker를 적용했습니다. 단순한 재시도 로직에 상태 추적과 강제 탈출 조건을 함께 설계해야 한다는 점을 이 구조를 통해 구체적으로 체득했습니다.

③ 환각 검증 강도의 트레이드오프

Evaluator의 is_grounded 검사는 생성된 답변이 retrieved_docs 내 근거에 기반하는지 검증합니다. 부동산·법률 도메인은 공문서 정책이 즉각적으로 변경되는 경우가 있어, LLM API가 내부적으로 검색을 거쳐 응답을 생성하더라도 해당 구조가 블랙박스인 이상 신뢰 근거를 시스템이 직접 통제할 수 없습니다. 오류 발생 시 피해가 크다는 점에서 grounding 검증의 도입 명분은 충분합니다. 다만 최신 LLM의 생성 품질을 고려할 때, is_grounded를 재시도 강제 트리거로 운용하면 오히려 정확한 답변을 탈락시키는 역효과가 생길 수 있습니다. 이에 따라 is_grounded를 hard block이 아닌 참고 플래그 수준으로 운용하고 최종 판단은 HumanReviewer에 위임하는 방식을 현재 검토 중입니다.

7-2. 현재 구조의 한계와 개선 방향

한계	개선 방향
Counselor의 단항목 순차 수집 방식으로 인한 대화 회전수 과다 및 사용자 응답 파싱 실패 빈번 발생	수집 전략(일괄 질문 vs 단계적 수집) 재설계 및 응답 파싱 로직 보완 시점 검토 필요
Vector + Graph 하이브리드 검색의 실 사용 성능 검증 필요	도메인 쿼리 유형별 두 검색기의 기여도 측정 및 가중치 조정
Planner가 지정 가능한 노드가 6개로 고정	도메인 확장 시 노드 추가와 Planner 프롬프트 함께 관리 필요
is_grounded 운용 강도 결정 미완료	hard block 또는 참고 플래그 중 도메인 리스크 허용 수준에 따라 결정

7-3. 핵심 설계 원칙 요약

- 인터페이스 분리 (**ExternalDepsPort**): 엔진과 인프라의 결합을 제거하여 각 레이어의 독립적 변경 가능
- 안전 우선 설계: **Circuit Breaker**, 3 중 품질 검사, **Human-in-the-Loop** 를 워크플로우에 내재화
- 점진적 확장 설계: **Optional** 기능(**GraphRAG**, 발화 **Vector DB**)을 명확히 분리하여 초기 복잡도 억제
- 단일 책임 노드: 각 노드는 하나의 역할만 담당하고 **AgentState** 를 통해 느슨하게 협력